

Linux Laboratory

Part 1 & Part 2

GIK2NV - Data storage and management technologies

Date: [DATE]

Name	DU-ID
Bilal [Last name]	[DU-ID]
Ali [Last name]	[DU-ID]
Merat [Last name]	[DU-ID]
Hussein [Last name]	[DU-ID]

Video presentation: [YouTube link]

Video presentation, division of work: Bilal – introduction and Part 1 tasks 1–3
· Ali – Part 1 tasks 4–5 · Merat – Part 1 task 6 and the Part 2 script code · Hussein
– running and testing the Part 2 script.

Laboratory environment

The lab was carried out on **Ubuntu 24.04 LTS** (hostname `linux-lab`). All administrative commands are run as `root` (on a normal user account, put `sudo` in front of each command). The users and groups below were used in Part 1:

Department	Group	Administrator	Users
Engineering	Engineering	eng_admin	eng_user1, eng_user2
Sales	Sales	sales_admin	sales_user1, sales_user2
HR	HR	hr_admin	hr_user1, hr_user2

Part 1 - User Management

Task 1: Create a directory at the root (/) for each department

We go to the root of the file system with `cd /` and use `mkdir` to create one directory per department, each named after its department. `ls -ld` verifies that the three directories exist directly under `/`.

```
root@linux-lab: ~  
root@linux-lab:/# cd /  
root@linux-lab:/# mkdir /Engineering /Sales /HR  
root@linux-lab:/# ls -ld /Engineering /Sales /HR  
drwxrwxrwx 2 root root 4096 Oct  9 12:29 /Engineering  
drwxrwxrwx 2 root root 4096 Oct  9 12:29 /HR  
drwxrwxrwx 2 root root 4096 Oct  9 12:29 /Sales
```

Task 2: Create a group for each department

`groupadd` creates one group per department, each named after its department. `getent group` verifies that the groups exist in `/etc/group`.

```
root@linux-lab: ~  
root@linux-lab:/# groupadd Engineering  
root@linux-lab:/# groupadd Sales  
root@linux-lab:/# groupadd HR  
root@linux-lab:/# getent group Engineering Sales HR  
Engineering:x:1001:  
Sales:x:1002:  
HR:x:1003:
```

Task 3: Create an administrative user for each department

`useradd` creates the three administrators:

- `-m` creates a home directory,
- `-s /bin/bash` gives the user a **Bash login shell** (3a),
- `-g <group>` sets the department group as the user's **primary group** (3b).

`id` shows that the primary group (`gid`) is the department group, and `/etc/passwd` shows that the login shell is `/bin/bash`.

```
root@linux-lab: ~  
root@linux-lab:~# useradd -m -s /bin/bash -g Engineering eng_admin  
root@linux-lab:~# useradd -m -s /bin/bash -g Sales sales_admin  
root@linux-lab:~# useradd -m -s /bin/bash -g HR hr_admin  
root@linux-lab:~# id eng_admin; id sales_admin; id hr_admin  
uid=1001(eng_admin) gid=1001(Engineering) groups=1001(Engineering)  
uid=1002(sales_admin) gid=1002(Sales) groups=1002(Sales)  
uid=1003(hr_admin) gid=1003(HR) groups=1003(HR)  
root@linux-lab:~# grep -E '^(eng|sales|hr)_admin' /etc/passwd  
eng_admin:x:1001:1001::/home/eng_admin:/bin/bash  
sales_admin:x:1002:1002::/home/sales_admin:/bin/bash  
hr_admin:x:1003:1003::/home/hr_admin:/bin/bash
```

Task 4: Create two additional users for each department

The same `useradd` options give each regular user a Bash login shell (4a) and the department group as primary group (4b). The output of `id` and `/etc/passwd` verifies this.

```
root@linux-lab: ~  
root@linux-lab:~# useradd -m -s /bin/bash -g Engineering eng_user1  
root@linux-lab:~# useradd -m -s /bin/bash -g Engineering eng_user2  
root@linux-lab:~# useradd -m -s /bin/bash -g Sales sales_user1  
root@linux-lab:~# useradd -m -s /bin/bash -g Sales sales_user2  
root@linux-lab:~# useradd -m -s /bin/bash -g HR hr_user1  
root@linux-lab:~# useradd -m -s /bin/bash -g HR hr_user2  
root@linux-lab:~# for u in eng_user1 eng_user2 sales_user1 sales_user2 hr_user1 hr_user2; do id $u; done  
uid=1004(eng_user1) gid=1001(Engineering) groups=1001(Engineering)  
uid=1005(eng_user2) gid=1001(Engineering) groups=1001(Engineering)  
uid=1006(sales_user1) gid=1002(Sales) groups=1002(Sales)  
uid=1007(sales_user2) gid=1002(Sales) groups=1002(Sales)  
uid=1008(hr_user1) gid=1003(HR) groups=1003(HR)  
uid=1009(hr_user2) gid=1003(HR) groups=1003(HR)  
root@linux-lab:~# grep -E '_user[12]' /etc/passwd  
eng_user1:x:1004:1001::/home/eng_user1:/bin/bash  
eng_user2:x:1005:1001::/home/eng_user2:/bin/bash  
sales_user1:x:1006:1002::/home/sales_user1:/bin/bash  
sales_user2:x:1007:1002::/home/sales_user2:/bin/bash  
hr_user1:x:1008:1003::/home/hr_user1:/bin/bash  
hr_user2:x:1009:1003::/home/hr_user2:/bin/bash
```

Task 5: Secure the department directories

5a - Owner is the department admin, group is the department group

`chown user:group` sets both the owner and the group owner in a single command. `ls -ld` shows the new owner and group.

```
root@linux-lab: ~  
root@linux-lab:~# chown eng_admin:Engineering /Engineering  
root@linux-lab:~# chown sales_admin:Sales /Sales  
root@linux-lab:~# chown hr_admin:HR /HR  
root@linux-lab:~# ls -ld /Engineering /Sales /HR  
drwxrwxrwx 2 eng_admin Engineering 4096 Oct 9 12:29 /Engineering  
drwxrwxrwx 2 hr_admin HR 4096 Oct 9 12:29 /HR  
drwxrwxrwx 2 sales_admin Sales 4096 Oct 9 12:29 /Sales
```

5b-5e - Permissions

All of the permission requirements are covered by one command, `chmod 1770:`

Digit	Meaning	Requirement
1	Sticky bit: only the owner of a file can delete it	5c
7	Owner (department admin): read, write, execute (full access)	5b, 5c
7	Group (department users): read, write, execute (full access)	5d
0	Others: no permissions at all	5e

In the `ls -ld` output, `drwxrwx--T` shows `rw` for owner and group and `--` for others. The `T` at the end is the sticky bit.

```

root@linux-lab: ~
root@linux-lab:/# chmod 1770 /Engineering /Sales /HR
root@linux-lab:/# ls -ld /Engineering /Sales /HR
drwxrwx--T 2 eng_admin  Engineering 4096 Oct  9 12:29 /Engineering
drwxrwx--T 2 hr_admin   HR          4096 Oct  9 12:29 /HR
drwxrwx--T 2 sales_admin Sales      4096 Oct  9 12:29 /Sales

```

Testing the directory permissions

- The admin (5b) and a regular user (5d) can both create files in the department directory.
- `eng_user2` cannot change or delete the file owned by `eng_user1`, but `eng_user1` can delete their own file. This shows the sticky bit works (5c).
- Users from other departments (`sales_user1`, `hr_admin`) cannot list or write to a directory that is not theirs (5e).

```

root@linux-lab: ~
root@linux-lab:/# su - eng_admin -c 'touch /Engineering/admin_file && echo "eng_admin: file created"'
eng_admin: file created
root@linux-lab:/# su - eng_user1 -c 'touch /Engineering/user1_file && echo "eng_user1: file created"'
eng_user1: file created
root@linux-lab:/# su - eng_user2 -c 'echo test > /Engineering/user1_file'
-bash: line 1: /Engineering/user1_file: Permission denied
root@linux-lab:/# ls -l /Engineering
total 0
-rw-r--r-- 1 eng_admin Engineering 0 Oct  9 12:29 admin_file
-rw-r--r-- 1 eng_user1 Engineering 0 Oct  9 12:29 user1_file
root@linux-lab:/# su - eng_user2 -c 'rm /Engineering/user1_file'
rm: cannot remove '/Engineering/user1_file': Operation not permitted
root@linux-lab:/# su - eng_user1 -c 'rm /Engineering/user1_file && echo "eng_user1: own file deleted"'
eng_user1: own file deleted
root@linux-lab:/# su - sales_user1 -c 'ls /Engineering'
ls: cannot open directory '/Engineering': Permission denied
root@linux-lab:/# su - hr_admin -c 'touch /Sales/hack'
touch: cannot touch '/Sales/hack': Permission denied
root@linux-lab:/# rm /Engineering/admin_file

```

Task 6: Create a document in each department directory

We create `confidential.txt` in each directory with `echo`, containing exactly one line of text (6b). `chown` gives the file the same ownership as its directory (6a). `chmod 640` sets the access (6c):

Digit	Who	Permission
6	Owner (department admin)	read + write: only the admin can modify the file
4	Group (department users)	read only
0	Others	no permissions

```
root@linux-lab: ~  
root@linux-lab:/# echo "This file contains confidential information for the department." > /Engineering/confidential.txt  
root@linux-lab:/# echo "This file contains confidential information for the department." > /Sales/confidential.txt  
root@linux-lab:/# echo "This file contains confidential information for the department." > /HR/confidential.txt  
root@linux-lab:/# chown eng_admin:Engineering /Engineering/confidential.txt  
root@linux-lab:/# chown sales_admin:Sales /Sales/confidential.txt  
root@linux-lab:/# chown hr_admin:HR /HR/confidential.txt  
root@linux-lab:/# chmod 640 /Engineering/confidential.txt /Sales/confidential.txt /HR/confidential.txt  
root@linux-lab:/# ls -l /Engineering /Sales /HR  
/Engineering:  
total 4  
-rw-r----- 1 eng_admin Engineering 64 Oct  9 12:29 confidential.txt  
  
/HR:  
total 4  
-rw-r----- 1 hr_admin HR 64 Oct  9 12:29 confidential.txt  
  
/Sales:  
total 4  
-rw-r----- 1 sales_admin Sales 64 Oct  9 12:29 confidential.txt  
root@linux-lab:/# cat /Engineering/confidential.txt  
This file contains confidential information for the department.
```

Testing the file permissions

- A department user can read the file but gets *Permission denied* when trying to modify it.
- The department admin can modify it.
- Users from other departments cannot read it.

```
root@linux-lab: ~  
root@linux-lab:/# su - eng_user1 -c 'cat /Engineering/confidential.txt'  
This file contains confidential information for the department.  
root@linux-lab:/# su - eng_user1 -c 'echo changed >> /Engineering/confidential.txt'  
-bash: line 1: /Engineering/confidential.txt: Permission denied  
root@linux-lab:/# su - eng_admin -c 'echo "This file contains confidential information for the department." > /Engineering/confidential.txt && echo "eng_admin: file modified"'  
eng_admin: file modified  
root@linux-lab:/# su - sales_user1 -c 'cat /Engineering/confidential.txt'  
cat: /Engineering/confidential.txt: Permission denied  
root@linux-lab:/# su - hr_user2 -c 'cat /Sales/confidential.txt'  
cat: /Sales/confidential.txt: Permission denied  
root@linux-lab:/# su - sales_user2 -c 'cat /Sales/confidential.txt'  
This file contains confidential information for the department.
```

Part 2 - User Management Script

How the script works

The script `usermgmt.sh` is interactive and contains **no hardcoded usernames or group names**. The administrator types in the group name, username and password while it runs. The steps are in a logical order: the group must exist before the user (it becomes the user's primary group), and the user and group must exist before the directory can be given to them.

The special variable `$?` holds the exit code of the previous command: `0` means success and anything else means failure. The script uses it after each important command to decide what to do next.

0. **Root check.** The script exits with an error if it is not run as root, since every step below needs root privileges.
1. **(a) Create a new group.** A `while true` loop asks for a group name with `read -p`. An empty name is rejected. `getent group <name>` looks the name up. If `$?` is `0` the group already exists, so an error is printed and the loop asks for another name. Otherwise `groupadd` creates the group, `$?` confirms it worked, and `break` leaves the loop.
2. **(b) Create a new user.** The same loop pattern with `getent passwd <name>` rejects existing usernames. A unique user is created with `useradd -m -s /bin/bash -g <group> <user>`, which gives a Bash login shell and makes the new group the primary group.
3. **(c) Create a password.** The password is read twice with `read -s`, so it is hidden while typing. An empty or mismatching password is rejected and the admin tries again. The password is set with `chpasswd`, which stores it as a hash in `/etc/shadow`.
4. **(d) Group membership.** `usermod -aG <group> <user>` adds the user as a member of the group, so they are listed in `/etc/group` as well as having it as their primary group.
5. **(e) Directory.** `mkdir /<username>` creates a directory at the root with the same name as the user. If this fails, the script stops with an error.
6. **(f) Ownership.** `chown <user>:<group> /<username>`.
7. **(g + h) Permissions.** `chmod 1770` gives full control to the owner (`7`) and the group (`7`) and nothing to others (`0`). The leading `1` is the sticky bit, which means only the owner of a file can delete it from the directory.
8. **Summary.** Prints `id`, the `/etc/passwd` entry and `ls -ld` of the directory as proof.
9. **(i) Executable.** The script is made executable with `chmod +x usermgmt.sh` and run as `./usermgmt.sh` (or `sudo ./usermgmt.sh`).

Script implementation

```
root@linux-lab: ~/lab
root@linux-lab:~/lab# cat usermgmt.sh
#!/bin/bash
# GIK2NV Linux Lab - Part 2
# Interactive user management script.
# Run as root: sudo ./usermgmt.sh

# The script must be run as root since it creates groups, users and directories
if [ "$(id -u)" -ne 0 ]; then
    echo "Error: this script must be run as root (use sudo)."
```

```
    exit 1
fi

# a. Create a new group - keep asking until a unique name is given
while true; do
    read -p "Enter a new group name: " groupname
    if [ -z "$groupname" ]; then
        echo "Error: group name cannot be empty. Try again."
        continue
    fi
    getent group "$groupname" > /dev/null
    if [ $? -eq 0 ]; then
        echo "Error: group '$groupname' already exists. Try another group name."
    else
        groupadd "$groupname"
        if [ $? -eq 0 ]; then
            echo "Group '$groupname' created."
            break
        else
            echo "Error: could not create group '$groupname'. Try another group name."
        fi
    fi
done

# b. Create a new user - keep asking until a unique name is given.
# Bash login shell and the new group as primary group.
while true; do
    read -p "Enter a new username: " username
    if [ -z "$username" ]; then
        echo "Error: username cannot be empty. Try again."
        continue
    fi
    getent passwd "$username" > /dev/null
    if [ $? -eq 0 ]; then
        echo "Error: user '$username' already exists. Try another username."
    else
        useradd -m -s /bin/bash -g "$groupname" "$username"
        if [ $? -eq 0 ]; then
            echo "User '$username' created."
            break
        else
            echo "Error: could not create user '$username'. Try another username."
        fi
    fi
done
```

```
root@linux-lab: ~/lab

root@linux-lab:~/lab# cat usermgmt.sh
# c. Create a password for the user
while true; do
    read -s -p "Enter a password for $username: " password
    echo
    read -s -p "Confirm password: " password2
    echo
    if [ -z "$password" ]; then
        echo "Error: password cannot be empty. Try again."
    elif [ "$password" != "$password2" ]; then
        echo "Error: passwords do not match. Try again."
    else
        echo "$username:$password" | chpasswd
        if [ $? -eq 0 ]; then
            echo "Password set for '$username'."
            break
        else
            echo "Error: could not set password. Try again."
        fi
    fi
done

# d. Ensure the new user is a member of the new group
usermod -aG "$groupname" "$username"
if [ $? -eq 0 ]; then
    echo "User '$username' is a member of group '$groupname'."
fi

# e. Create a directory at the root (/) with the same name as the user
mkdir "/$username"
if [ $? -ne 0 ]; then
    echo "Error: could not create directory /$username."
    exit 1
fi
echo "Directory /$username created."

# f. Set ownership of the directory to the new user and group
chown "$username:$groupname" "/$username"

# g + h. 1770: owner rwx, group rwx, others none, sticky bit set so that
#         only the owner of a file can delete it from the directory
chmod 1770 "/$username"
echo "Ownership and permissions set on /$username."

echo
echo "==== Summary ====="
id "$username"
getent passwd "$username"
ls -ld "/$username"
```

Testing

(i) Making the script executable

Before `chmod +x`, the file has no `x` bits (`-rw-r--r--`). Afterwards it is executable (`-rwxr-xr-x`).

```
root@linux-lab: ~

root@linux-lab:~/lab# ls -l usermgmt.sh
-rw-r--r-- 1 root root 3119 Oct  9 12:17 usermgmt.sh
root@linux-lab:~/lab# chmod +x usermgmt.sh
root@linux-lab:~/lab# ls -l usermgmt.sh
-rwxr-xr-x 1 root root 3119 Oct  9 12:17 usermgmt.sh
```

Test 1: Duplicate group, duplicate user, wrong password, then valid input

This run tests every error branch:

- **(a)** `HR` already exists, so the script reports an error and asks again. `Marketing` is accepted.
- **(b)** `eng_admin` already exists, so the script reports an error. `anna` is accepted.
- **(c)** The two passwords did not match the first time, so the script asks again. The second attempt matched. The password is hidden while typing.
- **(d-h)** The rest of the script runs, and the summary shows the result.

```
root@linux-lab: ~  
root@linux-lab:~/lab# ./usermgmt.sh  
Enter a new group name: HR  
Error: group 'HR' already exists. Try another group name.  
Enter a new group name: Marketing  
Group 'Marketing' created.  
Enter a new username: eng_admin  
Error: user 'eng_admin' already exists. Try another username.  
Enter a new username: anna  
User 'anna' created.  
Enter a password for anna:  
Confirm password:  
Error: passwords do not match. Try again.  
Enter a password for anna:  
Confirm password:  
Password set for 'anna'.  
User 'anna' is a member of group 'Marketing'.  
Directory /anna created.  
Ownership and permissions set on /anna.  
  
===== Summary =====  
uid=1010(anna) gid=1004(Marketing) groups=1004(Marketing)  
anna:x:1010:1004:~/home/anna:/bin/bash  
drwxrwx--T 2 anna Marketing 4096 Oct  9 12:29 /anna
```

Verification of Test 1

- `/etc/group` lists `anna` as a member of `Marketing` (a, d).
- `id` and `/etc/passwd` show that the primary group is `Marketing` and the shell is `/bin/bash` (b).
- `/etc/shadow` contains a password hash for `anna` (c).
- `/anna` exists at the root (e), is owned by `anna:Marketing` (f) and has permissions `drwxrwx--T` (g, h).
- `anna` can create files in it, and a user outside the group is denied access.

```
root@linux-lab: ~  
root@linux-lab:/# grep Marketing /etc/group  
Marketing:x:1004:anna  
root@linux-lab:/# id anna  
uid=1010(anna) gid=1004(Marketing) groups=1004(Marketing)  
root@linux-lab:/# grep anna /etc/passwd  
anna:x:1010:1004::/home/anna:/bin/bash  
root@linux-lab:/# grep anna /etc/shadow | cut -c1-30  
anna:$y$j9T$y8x6894v.Yv5e9UCRz  
root@linux-lab:/# ls -ld /anna  
drwxrwx--T 2 anna Marketing 4096 Oct  9 12:29 /anna  
root@linux-lab:/# su - anna -c 'touch /anna/anna_file && ls -l /anna'  
total 0  
-rw-r--r-- 1 anna Marketing 0 Oct  9 12:30 anna_file  
root@linux-lab:/# su - eng_user1 -c 'ls /anna'  
ls: cannot open directory '/anna': Permission denied
```

Test 2: A second run with other names

This run shows that the script accepts any names. It also tests an empty group name, and the group and user created in Test 1 are now rejected as duplicates.

```
root@linux-lab: ~  
root@linux-lab:~/lab# ./usermgmt.sh  
Enter a new group name: Marketing  
Error: group 'Marketing' already exists. Try another group name.  
Enter a new group name:  
Error: group name cannot be empty. Try again.  
Enter a new group name: Support  
Group 'Support' created.  
Enter a new username: anna  
Error: user 'anna' already exists. Try another username.  
Enter a new username: bob  
User 'bob' created.  
Enter a password for bob:  
Confirm password:  
Password set for 'bob'.  
User 'bob' is a member of group 'Support'.  
Directory /bob created.  
Ownership and permissions set on /bob.  
  
===== Summary =====  
uid=1011(bob) gid=1005(Support) groups=1005(Support)  
bob:x:1011:1005::/home/bob:/bin/bash  
drwxrwx--T 2 bob Support 4096 Oct  9 12:30 /bob
```

```
root@linux-lab: ~  
root@linux-lab:/# id bob  
uid=1011(bob) gid=1005(Support) groups=1005(Support)  
root@linux-lab:/# ls -ld /bob  
drwxrwx--T 2 bob Support 4096 Oct  9 12:30 /bob  
root@linux-lab:/# ls -ld /anna /bob  
drwxrwx--T 2 anna Marketing 4096 Oct  9 12:30 /anna  
drwxrwx--T 2 bob Support 4096 Oct  9 12:30 /bob
```

Conclusion

Part 1: every department has a private directory owned by its administrator and group. All department members have full access, files can only be deleted by their owner, and other users have no access. Each directory has a confidential file that only the administrator can modify and that department members can read.

Part 2: the script automates the same work for any group and user name. It validates input with `getent` and `$?` so duplicates are rejected, and it produces a correctly configured user and directory every time.